Alexander Matteson
ee513

Report: Project 1 ee513 (Characterize Your Camera)

Repository: AlexanderMatteson/Matteson_ee513_project1: Assignment 1 for ee513

See notebook ('PROJECT1_*ee515*_Matteson.ipynb') for functions and work.

Camera geometry. The camera is a projection of a 3D scene, and the interior of the camera can be explained by the focal length and the principal point (cx, cy):

Camera 1: Focal length = 1551.7 ± 1.4 px, 4.19 ± 0.02 mm. Principal point = (998.8 ± 0.87, 745.37 ± 1.1). Camera 2: Focal length = 794.8 ± 2.4 px, 1.549 ± 0.029 mm. Principal point = (1012.35 ± 0.8, 754.28 ± 0.72)

Compare: The principle point for both cameras in close to the center of their images. Camera 2 has a shorter focal length than camera 1. Apple says the camera 2 has a larger FOV (around 120 degrees). To fit a larger FOV on a given sensor we need to have a shorter focal length lens, which is consistent with the results here.

In real life, lenses are not ideal, therefore any lens will add consistent distortion and aberrations to the images which can be measured, modeled, and thus removed. We can correct for the distortions with the following equation:

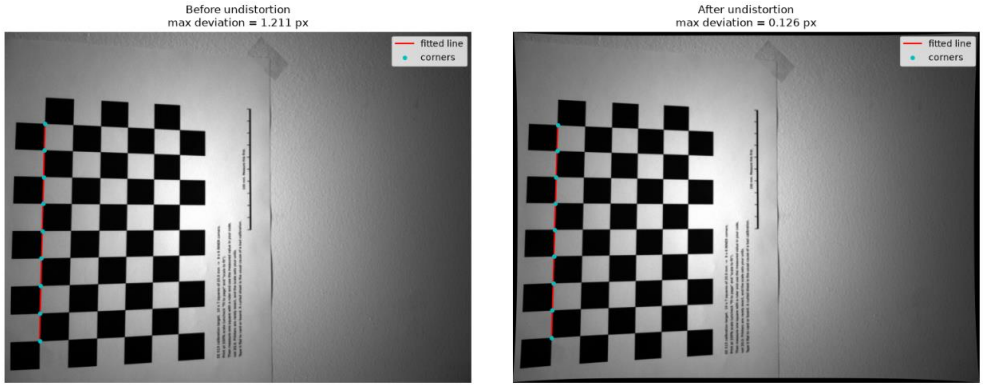
$$x_d = x_n \underbrace{\left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6\right)}_{\text{radial}} + \underbrace{2p_1 x_n y_n + p_2\left(r^2 + 2x_n^2\right)}_{\text{tangential}}$$

equation 1

Camera 1: k1 = 0.083 ± 0.0012, k2 = -0.149 ± 0.0028, p1 ± -0.0012, 0.00022, p2 = -0.00037, 0.0002.

Camera2: k1 = 0.1064 ± 0.0017, k2 = -0.1332 ± 0.0027, p1 = 0.00025 ± 0.0002, p2 = -0.00105 ± 0.00017.

Comparing: the wide field lens has a strong barrel distortion around the edges (see notebook). So while the lens in camera 2 gives a larger FOV it also causes straight lines going through the edges to bow out wards more. This affects the radial coefficients needed to correct the aberration.



Max perpendicular deviation: before = 1.211 px, after = 0.126 px

Fig 1: Checkerboard before and after distortion correction for camera 1. Measured perpendicular deviation of horizontal row of checkers before and after.

Method:

- Use open c v2.calibrateCamera function
- Capture 15 clean images (.dng raw files) of a checkerboard test target from different views (include captures with target along the edges and corners of the field of view)
- Report focal length, principal point, and radial and tangential distortion coefficients, include ± one standard deviation from the model as a measurement uncertainty on each number

Validation

Alexander Matteson
ee513

- Report the reprojection error from the caliration (camera 1 = 0.41); usually bellow 1 for a 'good' calibration, but a good RMS alone cannot tell you the model worked
- Check principal point in pixels is at the center of the image in pixels (camera 1 size (1512, 2016) so the reported principal point is near the center)
- Compare pixel pitch size calculated with two methods:
    - Method 1: using 1000 fmm/fpx (fmm = focal length from camera file, fpx = calibrated focal length)
    - Method 2: calculate pitch using the crop factor (does not rely on your calibration)
    - If the pixel pitch from these two methods do not agree this is your warning that something has gone wrong in the calibration

Spatial Resolution and the modulation transfer function (MTF) of our camera. The MTF is the magnitude of the Fourier transform of the point spread function; it is a measurement of how well the system transfers contrast at different spatial frequencies.

Camera 1: mtf50 center: 0.4002 ± 0.0108 cycles/px, mtf50 corner mean: 0.2927 ± 0.0196 cycles/px, mtf50 corner worst: 0.2704 ± 0.0099 cycles/px.

Camera 2: mtf50 center: 0.29 ± 0.055 cycles/px, mtf50 corner mean: 0.27 ± 0.034 cycles/px, worst corner mtf50: 0.236 ± 0.025 cycles/px.

Compare: the MTF50 for the second camera is much lower. Even though both cameras should be sampling limited, the MTF of the system is a product of the MTF of the parts. The lens of the second camera should have a higher f number according to the file properties, so it will have a lower spatial resolution cut off.

Method:

- ISO 12233 slanted edge
- Capture a slanted edge, linear (raw, .dng file)
- Find the edge position in each row to sub-pixel precision
- Fit a line through those positions, so one bad row cannot hurt you
- Project every pixel onto the edge normal, bin at about 1/4 pixel
- That is the edge spread function; differentiate it for the line spread function
- Window it, take the magnitude of its Fourier transform, normalize to 1 at DC That is the MTF
- Repeat for 5 captures and on each of the 5 positions in each capture target (so we can report a standard deviation for each measurement)

Validation:

- generate synthetic edge
- Measure the MTF and MTF50 on that synthetic edge using the measurement code
- Compare result to known analytic values for the synthetic edge
- Report if our method returns a value that agrees with the analytic value

Validation Result:
The analytic MTF50 = 0.1874 cycles per pixel
The measured MTF50 = 0.1846 cycles/pixel
The percent error = 1.4646661372169811%

Noise: poison gaussian model. We measure the inside of our sensor through a model of noise obtained using flat diffused white raw images. This gives us a measurement of the conversion gain and read noise.

$$\mathrm{Var}[D] = \underbrace{K}_{\text{slope}}(\mathbb{E}[D] - B) + \underbrace{K^2\sigma_r^2}_{\text{intercept}}$$

Equation 2

The slope (K) of equation 2 gives us the conversion gain in DN per e-. The intercept gives the read noise ().

Alexander Matteson
ee513

Camera 1: conversion gain (K) = 0.4099 ± 0.004 DN /e-, read noise = 2.8 ± 1.05 e-, dark current = 0.611 ± 0.113 e-/s, iso of noise measurement = 32, black level = 527.729 ± 0.044 DN.

Camera 2: Conversion gain (k) = 0.7215 ± 0.008 DN/e-, read noise = 3.20 ± 0.86 e-, dark current = 0.244 ± 0.080 e-/s, iso for noise measurements = 32.

Compare: camera 2 has a notably higher conversion gain (K) than camera 1. The conversion gain comes from the gain applied when going from electrons to digital numbers. The sensor for camera 2 has smaller pixels, therefore the conversion gain will be higher because the smaller pixel collects fewer phones and will then convert the fewer electrons to a higher count. The smaller pixels may have been chosen to account for the wide field lens having a shorter focal length.

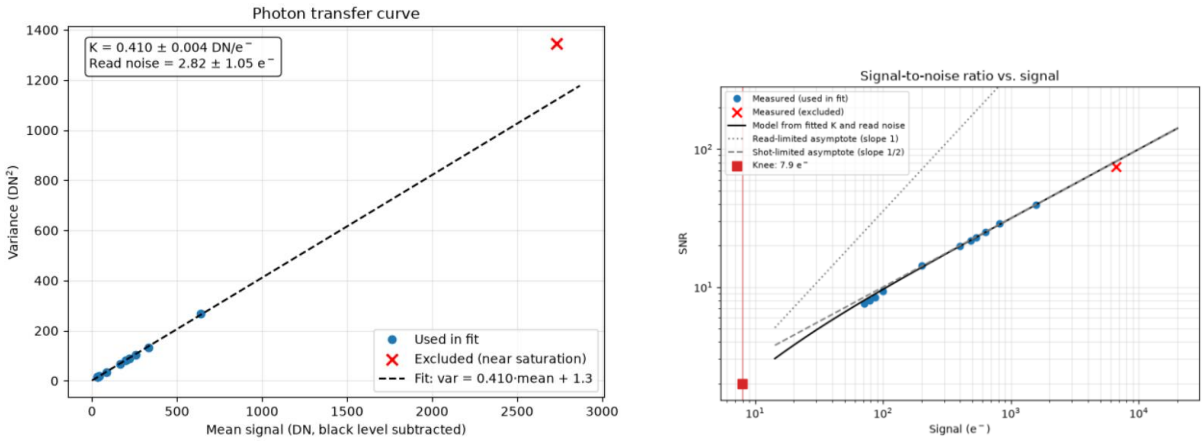


Figure 2: Camera 1 photon transfer curve, noise model fit to the linear portion of our white field data. Plotting signal to noise ratio vs signal on log scale, our data falls exclusively above the knee in the shot noise domain.

Methods Photon Transfer Function:

1. acquire 10+ pairs of images (lens covered with printer paper of a white background)
2. pairs are acquired because taking the difference allows us to remove the fixed pattern noise in all images
3. the images should span a range of exposure values
4. mean signal of each pair
5. variance of each pair
6. plot mean vs variance
7. fit a line to the linear region (report slop and intercept from the fir as well as uncertainties from the fit on all numbers)

Validation:

1. Generate camera sensor whit field data images (with a known gain and read noise it used to generate the frames)
2. Calculate the gain and read noise from the simulated flat white images using my photon transfer function code
3. Compare calculated gain and read noise values to the true values used to create the images as a percent error

Manufacturer values:

1. get the slope and intercept from the raw captures file properties
2. compare the manufacturer model results curve to our calculation from the white field captures dataset.

Validation Results:

Alexander Matteson
ee513

Simulated pairs K = 0.4236 DN/e-
Simulated pairs read noise = 2.9602 e-
true K = 0.4266 DN/e- and read noise = 2.80 e-
K percent error: -0.6900710419780219%
read percent error: 5.721657632477197%

Our fit photon transfer curve deviates largely from the manufacturer. This does not necessarily mean that our fit is incorrect. See the attached notebook for further discussion.

Dynamic range: the dynamic range for camera 1 is 11.11 ± 0.43 stops. This value is measured by the ratio of the largest signal we can measure over the smallest. The largest signal is defined as the full well value, which is 7419.76 ± 210.58 e-; this value is the max number of e- a pixel can hold without becoming saturated. The smallest signal we can measure I defined as where SNR = 1 because this will include both read and shot noise. The uncertainties on the measurements come from the fit covariance.

Camera 2 dynamic range:
Full well: 2769.79 ± 30.71 e-
read noise: 3.20 ± 0.86 e-
DR: 9.53 ± 0.36 stops
Compare: camera 1 has a higher dynamic range than camera 2. This seems to be a result of the full well value being much larger for camera 1. This makes sense because according to the spec sheets, online camera 2 has smaller pixels, so each pixel collects fewer light photons.

Color accuracy.
Method:

- Capture 3 images of color charts and a white background under four different illuminations
- Linear data
- Lens shading correction
- White balance first (von Kries methos scales each channel independently)
- Using least squares, fit the 3x3 white balanced camera RGB to XYZ
- Report CIEDE2000 per patch, before and after

Results (only for camera 1):
Metric: CIEDE2000 (dE00). Uncertainty = sample standard deviation across the 3 charts.
Mean dE00 after correction: 2.815 ± 0.076 (before: 8.55 ± 0.12)
Worst-patch dE00 after correction: 8.937 ± 0.045 (before: 14.47 ± 0.28)



Figure 3: Color chart capture and after color correction matrix was applied

Alexander Matteson
ee513

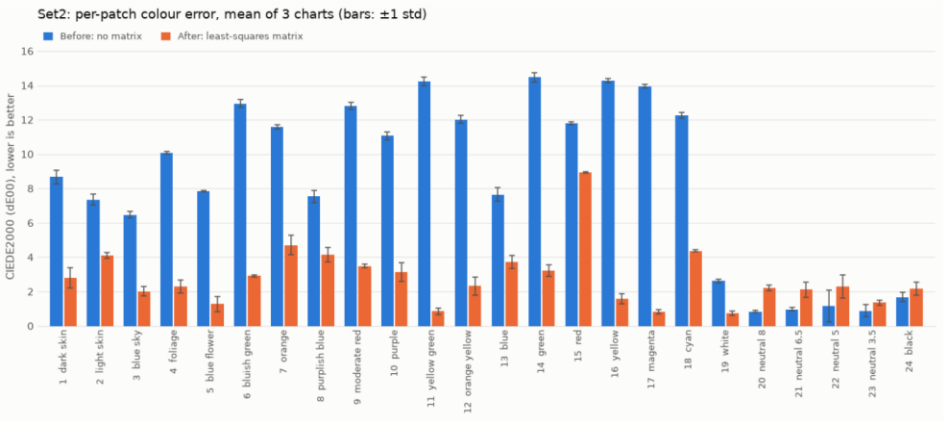


Figure 4: thee before and after mean dE00 values for three color maps shot under illumination condition 2. Color correction matrix calculated by least squares fitting.

One Page Decisions Document:

First on AI tools, I used claude ai to help write much of the code. This was especially useful for understanding libraries such as open CV that I had not used before and for making figures. Without ai coding tools these steps would have taken alot longer to look up and read documentation for. Another step that this was helpful was for cleaning up code into cleaner functions and catching syntax errors; both can be very time-consuming steps.

For any measurement numbers produced by code written by either ai or me, it is important to include verifications and checks against standards or known values wherever possible. For the calculation of the pixel pitch in mm, we can calculate it in two ways. One from the calibration using the checkerboard and again from the crop factor of the camera that is known and compares the percent difference between the two. My difference was 2.62774%. This shows a good agreement between the two. Checking that our calibrated values could be used to undistort a checkerboard near an edge was a nice visual that the process is working.

For the first camera we made the decision to fic k3 of the radial distortion coefficients to 0, this was done as this coefficient corrects for higher order effects of lens curvature and we know that it should be 0 for the first camera. Therefor it is better to fix it at 0 than allow the model to try to estimate it. However, for the second camera we choose not to fix this value. The second camera has a much stronger curvature from the wide-angle lens. Our fit does much better correct the barrel distortion effects when we include k3 in our model.

To validate the code written to measure the MTF from a slanted edge we generated a synthetic edge with a known gaussian noise added. Then measure the MTF and MTF50 on the synthetic edge using code. This allows us to check the agreement of our code against a known analytic solution. For the calculation of the photon transfer function, we can verify our code using generated camera sensor data white images with a known gain and read noise. Then we can calculate the gain and read noise using our photon transfer function code and compare the result to the known values. Our code produces a percent error for the gain (-0.69%), and a percent error for the read noise (5.72%).

The manufacturer also includes noise model numbers with the captures; this gives us another value to compare our results to. Our K gain was 0.4099 ± 0.0038 DN/e- which was larger than the manufacturer value of 0.1014 DN/e-. This then results in a smaller read noise value for our measurements than the manufacture. I guess this surprised me as I was feeling good about mine, given we had tested our code on the simulated data and comparing the calculated and true values for K and read gave small percent error values. I believe that this means that our code is not likely the issue here. This means the discrepancy would be from either my captures or differences/errors in the vendor calculation.

Alexander Matteson
ee513

Possible errors in my captures: one would be maybe our flat white fields are unevenly illuminated, or maybe there was movement between the pairs of captures increasing the variance too much, or maybe I had different iso between the captures, or maybe some pairs do not have matching apertures. The ISO and aperture concerns as well as any concerns of not using the correct camera should be caught by checks in the code above, so I rule this possibility out. The other two problems should be mitigated by the fact that I use only a middle patch of the whit field illumination so we should not be effected by vignetting and that I used a sort of homemade stand (out of books) to keep the camera in the same place while measuring (however this is relatively crude set up so it could certainty not be perfectly stable).

Looking into how good our fit is; plotting the residuals they seem to be randomly around 0. This is a good sign that my fit is ok. For possible assumptions of the manufacture result that may account for the difference include, we can't confirm that theirs is for the same iso or different gain than us, maybe they do other processing on their images before calculation the fit, and they also do not report an uncertainty on their measurements.

Resume Line: Built a tool kit to characterize the cameras on a smart phone (camera geometry, MTF, noise model, and color accuracy), published the results of characterizing the two cameras on the back of iPhone mini 12 and open-source code to GitHub.